Cleaning Manager: Authorization Across Component Boundaries

Software Security, Assignment I

Marcus Mitelea, 202511022
Aarhus University

October 2026

Repository: `https://github.com/mitmarcus/AU_SoftSec`, folder `Assignment_1`. All paths below are relative to that folder, and line numbers refer to tag `v2-fixed` (`9661bad`) unless I say otherwise.

1 System and component boundaries

I built a small back end for a cleaning company, modelled on CleanManager, a Danish system that cleaning firms use to plan jobs and pay cleaners. A manager assigns jobs, cleaners clock in and out on their own jobs, and a cleaner generates a payslip for a month from the registered hours. I picked this domain because it has data that clearly belongs to one person (hours, hourly rate, payslip), a privileged role that needs to see everyone's data, and a processing step (calculating pay) that fits a separate service and an external package. The seed data has two cleaners, `iza` (180 DKK/h) and `marius` (210 DKK/h), and one manager, `andreea` (`seed/seed.py`).

The application has three services, each a FastAPI app in its own container (`docker-compose.yml`). Figure 1 shows them.

- **api** is the only service with a published port (8000). It logs users in, mints tokens and forwards requests. It stores nothing.
- **records** owns all data: the SQLite file with users, jobs and time entries, and the payslip CSVs on the `records-data` volume. It checks every request itself.
- **payroll** runs one payslip job. It reads entries from `records`, calculates pay with the helper package `payrates` and writes the payslip back. It has no key, no volume and no database (`docker-compose.yml:26-31`).

The four functions are: list and search your own jobs (a manager sees and assigns all jobs); clock in and out on your own job; generate and download your own payslip for a month; and, for the manager only, list all cleaners' hours and payslips. The first, second and fourth go `api` → `records`. Generating a payslip goes `api` → `payroll` → `records` and is the one function that uses all three components.

I split the work this way so that the service running third-party code (`payroll`) is also the one with the least power, and so that the service holding the data (`records`) does not have to trust the other two. A simpler split would have let `api` do all authorization and treat `records` as a dumb store. I rejected that because then any process that can reach `records` on the internal network can read everything, and `payroll` has to reach `records` to do its job. Keeping the checks next to the data means a mistake in `api` or a compromised `payroll` still runs into the same ownership rules.

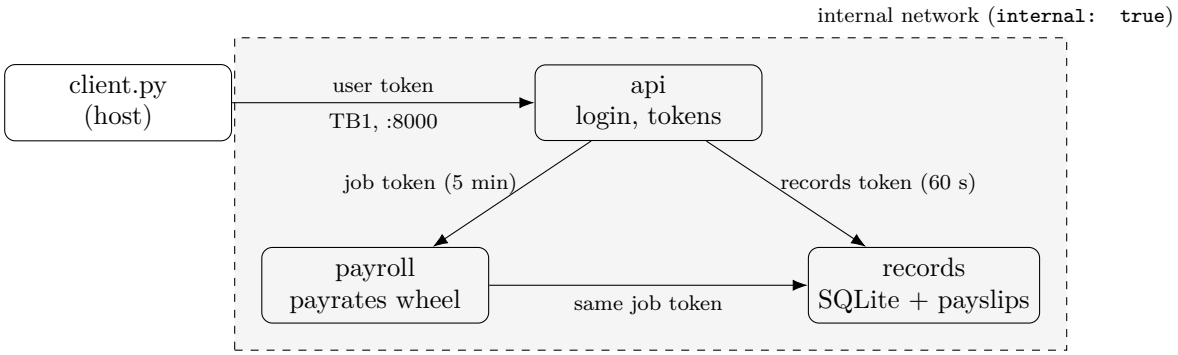


Figure 1: Components and the token on each edge. TB1 is the boundary to the host; every service-to-service edge is a boundary that `records` checks.

Where identity, authority, ownership and dependency trust are established

Identity is established once, at login. `records` compares the password with a bcrypt hash (`records/app.py:110-115`), and `api` mints a user token with `sub`, `role`, `aud=api` and a 30-minute expiry (`api/app.py:61-65`). After that, the identity is whatever the verified `sub` claim says. No service reads a user name from the request body or a header.

Authority is the role in the user token plus, for a payslip job, the scope list in the job token. `api` checks the role before forwarding (for example `api/app.py:121` for payslips, `:147` for the manager overview). `records` checks it again.

Ownership is established only in `records`, because it is the only place that knows who owns a job, an entry or a file. Every query filters on the token's `sub`: `WHERE cleaner = ?` for jobs (`records/app.py:128-145`), `own_job` for clocking (`:165`), and the cleaner's own folder for payslips (`:235-239`).

Dependency trust is established at build time. The `payrates` wheel is pinned by SHA-256 in `payroll/payrates.txt` and installed by pip in hash-checking mode, without resolving dependencies (`payroll/Dockerfile:9`). The pipeline also checks the hash before building. I tested the gate with a wheel that had one extra byte: pip refused it with "THESE PACKAGES DO NOT MATCH THE HASHES", expecting `703d50fb...` and getting `451130ac...` (`evidence/wheel-tamper.log`).

Design principles

I applied the three principles the assignment names. Each one maps to code and to a test in `tests/test_authz.py` that passes on `v2-fixed` (`evidence/v2-fixed/junit.xml`).

2 How authority travels through the payslip workflow

There are three kinds of token, and the difference between them is the main idea of the design. A *user token* (`aud=api`, 30 min) proves who the client is. For every user request, `api` mints a *records token* (`aud=records`, 60 s) with the same `sub` and `role`, so `records` can make its own decision instead of trusting that `api` already checked (`api/app.py:40-44`). For a payslip, `api` mints a *job token* (`aud=records`, 5 min) with `sub`, `job_id`, `month` and `scope=["entries:read","payslips:write"]` (`api/app.py:125-131`).

Together the claims bind the authority to four things the assignment asks for:

- **Service:** `aud`, and a separate HMAC key per audience. A token signed with the `api` key is rejected by `records` (`test_records_rejects_token_signed_with_api_key`).
- **Resource:** `sub` and `month`.

Principle	Implementation	Test
Least privilege	`payroll` gets no key, no volume and only the internal network (`docker-compose.yml:26-31`). Its job token covers one cleaner, one month, two scopes and 5 minutes (`api/app.py:125-131`).	`test_payroll_cannot_write_other_month`, `test_records_rejects_missing_scope`
Complete mediation	Every `records` route depends on `claims`, which verifies signature, audience, expiry and required claims (`records/app.py:60-68`), whether the caller is `api` or `payroll`.	`test_records_rejects_expired_job_token`, `_wrong_audience`, `_token_signed_with_api_key`
Fail-safe defaults	A missing header defaults to `""` and gives 401 (`records/app.py:60-63`). An unknown role gives 403 (`:71-75`). A token without a `scope` claim is treated as having none (`:91`).	`test_no_token_is_rejected`, `test_job_token_cannot_use_user_endpoints`, `test_cleaner_cannot_call_manager_endpoints`

Table 1: Design principles, where they are in the code, and the tests that check them.

- **Action:** `scope` for job tokens, `role` for records tokens.
- **Validity period:** `exp`, which PyJWT checks by default.

I considered forwarding the user token to `records` as it is. I did not, for two reasons. First, its audience is `api`, so accepting it in `records` would mean a token issued for one service works at another, which is the replay the `aud` claim exists to stop. Second, it lives for 30 minutes; the records token lives for 60 seconds and exists only for one call. The cost is that `api` holds the records key, so `api` is trusted to state the user's identity truthfully. It is not trusted to decide ownership.

A records token has a role and no scope; a job token has a scope and no role. So neither works on the other's endpoints (`records/app.py:71`, `:90`). I use HS256 because all keys stay inside one compose project; Section 4 says what I would change for production.

Figure 2 shows the payslip flow and where each check happens.

What stops payroll, a permitted service, from overstepping

`payroll` is allowed to read entries and write payslips, so the question is what keeps it inside one job. Four things do.

1. **It cannot change its authority.** It holds no signing key, so it cannot mint a token or edit the one it got. Any change breaks the signature, and `records` returns 401.
2. **The cleaner comes from the token only.** `records` uses `who = c["sub"]` and ignores headers and body fields (`records/app.py:212`, `:224`). `test_payroll_cannot_act_for_other_cleaner` sends a valid job token for iza together with `X-Cleaner-Id: marius`, and gets iza's data and `iza-2026-09.csv`.
3. **Scope and month are checked on each call** (`records/app.py:90-94`). A job token for September cannot write October (`test_payroll_cannot_write_other_month`). A token with only `payslips:write` cannot read entries (`test_records_rejects_missing_scope`).
4. **The token expires after 5 minutes** (`test_records_rejects_expired_job_token`), and the container has nowhere else to send data: `internal: true` gives it no route out (`docker-compose.yml:44`).

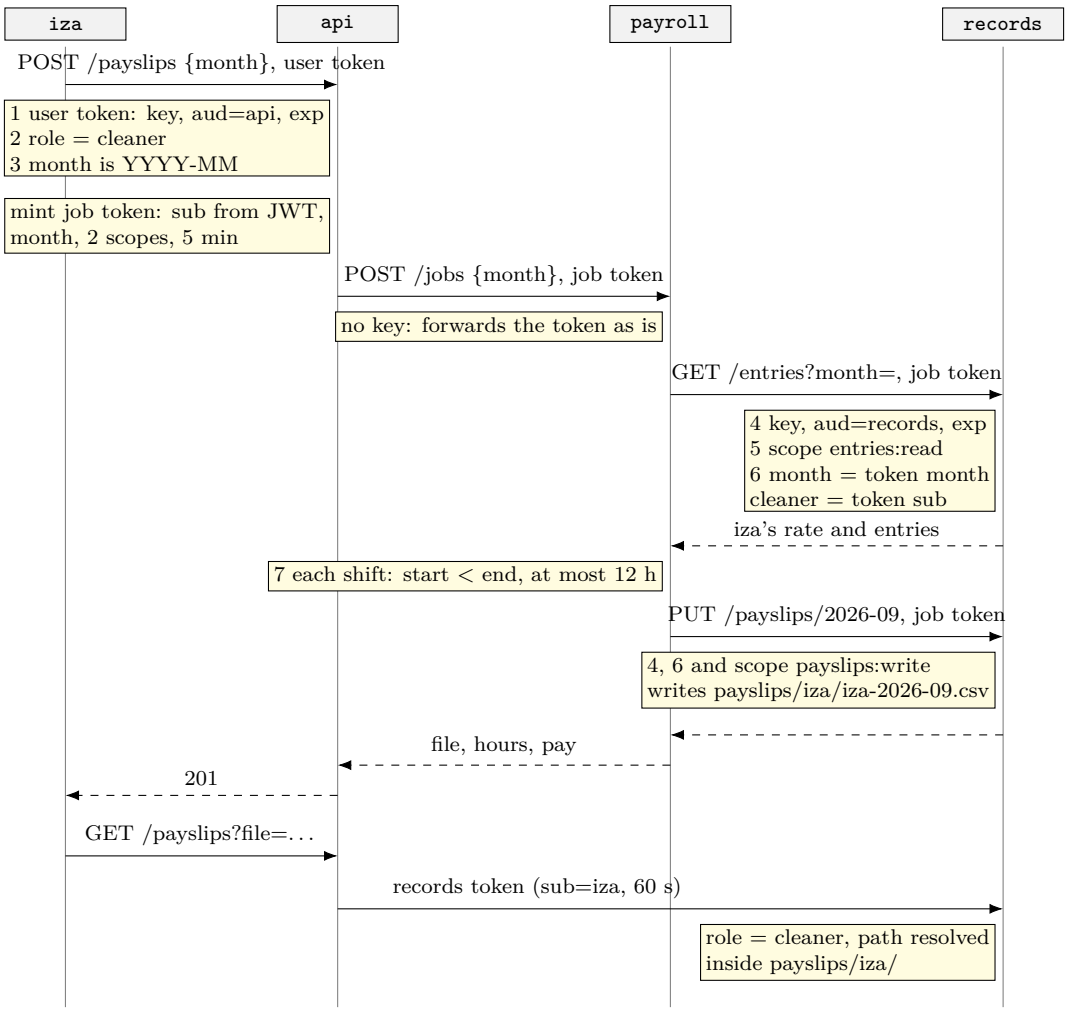


Figure 2: Payslip generation and download. The numbered boxes are the checks; `payroll` cannot change any token it carries.

`payroll` also does not trust `records` completely. Since the CWE-20 fix it rejects any shift that does not end after it starts or that lasts more than 12 hours, before it calculates pay (`payroll/app.py:50-52`).

3 Supply chain

I did not do the optional supply-chain attack or Experiments A and B. The only supply-chain control is the build-time hash pin from Section 1, and it is tested only by the tampered-wheel run in `evidence/wheel-tamper.log`.

4 How the plan shaped development

I committed the lifecycle plan (`docs/PLAN.md`) in `65be63d`, before any feature code (the first service code is `f6b5d4c`). It fixed the token design, the access-control matrix and the four planned vulnerabilities in advance. That turned out to matter most for the composition bug: the matrix says `payroll` acts on "`sub` and `month` of token", so two authorization tests written in Stage 3 (`aa31756`) already encoded that rule before the bug existed.

The plan also decided how I test. Several rules can only be checked by calling `records` directly, for example that an expired or wrongly scoped job token is rejected, but `records` has

no published port. Instead of opening a port for testing, I run the API tests in a `tests` container on the internal network (`tests/Dockerfile`, run by `make test`). The tests mint their own job tokens with the records key (`tests/conftest.py`) and play the part of `payroll`. So the authorization tests attack the same interface a compromised `payroll` would have.

Running the pipeline in a real Jenkins also changed things. Mounting `tests/` into the container broke, because Jenkins talks to the host's Docker daemon and the mount path only exists inside the Jenkins container. The tests are now copied into the image (`e2eeee0`). ruff 0.17 also flagged FastAPI's `Depends()` defaults as B008, so I added an allow-list in `ruff.toml` (`ff7cd36`) instead of turning the linter off.

Getting the Jenkins evidence itself took longer than writing the pipeline. I ran Jenkins in a container on my laptop and worked on the project over several days on different Wi-Fi networks. Docker writes a container's DNS server into it when the container starts, so after each network change the Jenkins container still pointed at the old network's DNS server. The first time, the setup wizard could not reach `updates.jenkins.io` and installed 0 plugins. Later, a branch scan hung because it could not resolve `api.github.com`, so a push was not built until I restarted the container. One build of `main` (build 7, on commit `7a0ca4c`) failed with 15 errors because the test container could not resolve `api` right after a network change; build 8 of the same commit passed all 25 tests (`evidence/jenkins/main-build-7/`, `main-build-8/`). In the end I ran the Jenkins container in Docker's host network mode, so it uses the laptop's own resolver, which follows network changes. Sadly I also had to reset Jenkins itself several times while developing, which deleted its build history, so the older builds are lost. The runs on both tags in `evidence/v1-vulnerable/` and `evidence/v2-fixed/` come from a separate Jenkins instance with the same `Jenkinsfile`. None of this was a problem in the application, but it shows that CI results are only as reliable as the machine they run on, and a shared build server would avoid it.

When I planted V4 in `baed1bb`, those two tests failed: `test_payroll_cannot_act_for_other_cleaner` and `test_api_ignores_cleaner_in_payslip_request`. That failure, not static analysis, is what blocked the vulnerable build. The gate decisions were:

Gate	Result	Evidence
Requirements	Plan reviewed before code; roles and architecture unchanged afterwards	`65be63d`
Build	Wheel hash OK; tampered wheel refused	`evidence/v2-fixed/pipeline.log`, `wheel-tamper.log`
Testing, `v1-vulnerable`	Blocked: 2 failed, 23 passed, "Finished: FAILURE"	`evidence/v1-vulnerable/pipeline.log`
Testing, `v2-fixed`	Passed: 25 + 3 tests, no bandit HIGH, 0 vulnerabilities in 24 audited packages	`evidence/v2-fixed/pipeline.log`

Table 2: Gate decisions from the Jenkins runs on both tags.

Two results did not match the plan. First, the blocking rule "any bandit HIGH" did not catch the SQL injection. Bandit reported it as B608 with MEDIUM severity and LOW confidence at `records/app.py:143` (`evidence/v1-vulnerable/bandit.json`). I kept the rule and recorded

the gap instead of lowering the threshold after seeing the result. Second, the path traversal and SQL injection were not caught by any Stage 3 test, because those tests did not try `../` or quote characters. Only the exploit tests show them. A test suite checks the rules I thought to write down, not every input.

Where each protection comes from

- **Language and runtime.** Python has no manual memory management, so the memory-safety weaknesses in the catalog (CWE-787, CWE-416) do not apply. `pathlib`'s `resolve()` and `is_relative_to()` do the canonicalisation for the CWE-22 fix. `sqlite3` placeholders keep data out of SQL syntax, and `execute` runs only one statement, so stacked queries fail even in the vulnerable version.
- **Libraries.** PyJWT checks signature, `exp` and `aud`, and the `require` option rejects tokens with missing claims. bcrypt handles password hashing. Pydantic rejects request bodies with the wrong types, for example a clock time that is not a datetime. Pydantic only checks syntax, not meaning, which is why CWE-20 was still possible.
- **Application logic.** Role, ownership, scope and month checks in `records`; role checks repeated in `api`; shift validation in both `records` and `payroll`.
- **Deployment configuration.** Only `api` is published. `records` and `payroll` sit on an `internal: true` network. Every container runs as a non-root user (`USER app` in each Dockerfile). `payroll` has no volume, and the wheel is installed by hash.

What a production release would need

This is a coursework deployment, and several things are deliberately simple.

- **Secrets management.** The JWT keys have development defaults in `docker-compose.yml:12-13`. In production they would come from a secret store (for example Docker secrets or Vault) and never appear in the repository.
- **TLS between services.** All traffic is plain HTTP, including bearer tokens on the internal network. I would use TLS at the edge and mutual TLS between services, so that `records` also knows which service is calling and not only which token it carries.
- **Key rotation.** One static key per audience means a leaked `RECORDS_JWT_KEY` lets anyone mint job tokens for any cleaner. I would switch to asymmetric keys (ES256): `api` signs, `records` verifies with a public key published with a `kid`, so keys can rotate without redeploying `records`.
- **Logging and monitoring.** There is no audit log. I would log every authorization decision in `records` (who, what, allowed or denied) and alert on repeated 403s. Repudiation is currently covered only by storing the creator's `sub` on each entry.
- **Signed artifacts.** The wheel hash proves the bytes did not change after I approved them, not who built them. I would sign the wheel and the container images (for example with Sigstore) and verify the signatures at deploy time.

Rate limiting on login and the internal `/login` endpoint, which any container on the network can call, would also need attention.

5 Planted vulnerabilities

I planted four vulnerabilities, each in its own commit with a `# VULN` comment, and showed each one with a test in `tests/exploits/`. Tag `v1-vulnerable` (`305067f`) contains all four, and `v2-fixed` (`9661bad`) contains all the fixes. The full analysis, in the catalog's six sections, is in `docs/VULNERABILITIES.md`.

CWE	What happened	Planted	Fixed
CWE-22	The payslip file name was joined onto the cleaner's folder without canonicalisation; iza downloaded `../marius/marius-2026-09.csv`	`405b6f1`	`4853573`
CWE-89	The cleaner's job search built SQL with an f-string; a search text that closes the quote and appends `OR 1=1` and an SQL comment returned every job	`9ef01b7`	`00f06ef`
CWE-20	Clock-out accepted any end time and `payroll` paid it; a 30-hour shift added exactly 30 hours to iza's payslip	`c929555`	`552b4d8`
CWE-862 (composition)	`api` took `cleaner_id` from the body, `payroll` sent it as `X-Cleaner-Id`, `records` trusted it; iza got marius's payslip with his 210 DKK rate	`baed1bb`	`9661bad`

Table 3: Planted vulnerabilities with the commits that added and removed them.

The composition bug is the one I learned most from. Seen alone, each service looks correct: `api` authenticates, `payroll` forwards, and `records` verifies the job token completely. The bug only exists because the cleaner crossed the services in a plain field next to the token, and `records` treated being on the internal network as being trusted. The fix removes that field and binds the cleaner to the signed `sub` only. My takeaway is that every piece of authority has to travel inside something the receiver can verify; anything next to the token is just input.

The other three taught smaller lessons. The CWE-22 fix shows that choosing the folder from the token is not enough when the file name can still leave it; the check has to be on the final, resolved path. Exploiting it also needed a precondition I did not expect: Linux only resolves `iza/..` if `payslips/iza/` exists, so iza first had to generate her own payslip. The CWE-89 case was the only one that static analysis noticed at all. The CWE-20 fix is the only one where I added a check in a second service: `records` validates at the point of entry, and `payroll` validates again before paying, because the catalog names "assuming another component has already validated the input" as the underlying factor. No test reaches the `payroll` check alone, since `records` now rejects bad shifts first; I state that gap in `docs/VULNERABILITIES.md`.

A Evidence index

Item	Location
Requirements, plan, decisions	`docs/REQUIREMENTS.md` (`4190cdf`), `docs/PLAN.md` (`65be63d`), `docs/DECISIONS.md`
Architecture, threat model, access control	`docs/ARCHITECTURE.md`, `docs/THREAT_MODEL.md`, `docs/ACCESS_CONTROL.md`
Pipeline definition	`Jenkinsfile` (`305067f`), `jenkins/Dockerfile`
Pipeline runs	`evidence/v1-vulnerable/`, `evidence/v2-fixed/` (console log, `bandit.json`, `pip-audit.json`, `junit.xml`)
Hash gate	`evidence/wheel-tamper.log`
Tests	`tests/test_functional.py` (7), `tests/test_authz.py` (13), `tests/exploits/` (4), `payrates/tests/` (3)